

Read-Write Locks (RW Locks)

What are Read-Write Locks?

Read-Write Locks are synchronization mechanisms that allow:

Multiple Readers

OR

Single Writer

at a time.

They improve performance when data is read frequently and written rarely.

Why Do We Need RW Locks?

Suppose a shared resource is accessed as follows:

95% Reads

5% Writes

Using a normal mutex/spinlock:

Reader1 -> Read

Reader2 -> Wait

Reader3 -> Wait

Even though readers are only reading data, they block each other.

RW Locks allow multiple readers to access data simultaneously.

Basic Principle

Multiple Readers Allowed

Reader1 -> Reading

Reader2 -> Reading

Reader3 -> Reading

All readers can access data concurrently.

Single Writer Allowed

Writer1 -> Writing

Writer2 -> Waiting

Only one writer can modify data.

Readers and Writers Cannot Execute Together

Writer -> Writing

Reader1 -> Waiting

Reader2 -> Waiting

OR

Reader1 -> Reading

Reader2 -> Reading

Writer -> Waiting

Types of Read-Write Locks

Linux provides two types:

1. RW Spinlock
2. RW Semaphore

1. RW Spinlock

Header File:

```
#include <linux/rwlock.h>
```

Declaration:

```
rwlock_t rwlock;
```

Characteristics:

Busy Waiting

No Sleeping

Short Critical Sections

Interrupt Context Supported

APIs

Initialization

```
rwlock_init(&rwlock);
```

Reader Lock

```
read_lock(&rwlock);
```

```
/* Read Critical Section */
```

```
read_unlock(&rwlock);
```

Writer Lock

```
write_lock(&rwlock);
```

```
/* Write Critical Section */
```

```
write_unlock(&rwlock);
```

2. RW Semaphore

Header File:

```
#include <linux/rwsem.h>
```

Declaration:

```
struct rw_semaphore rwsem;
```

Characteristics:

Sleeping Lock

Long Critical Sections

Process Context Only

APIs

Initialization

```
init_rwsem(&rwsem);
```

Reader Lock

```
down_read(&rwsem);
```

```
/* Read Critical Section */
```

```
up_read(&rwsem);
```

Writer Lock

```
down_write(&rwsem);
```

```
/* Write Critical Section */
```

```
up_write(&rwsem);
```

RW Spinlock vs RW Semaphore

RW Spinlock	RW Semaphore
Busy Wait	Sleep
No Sleeping Allowed	Sleeping Allowed
Short Critical Section	Long Critical Section
Interrupt Context	Process Context
Faster	Higher Overhead

Advantages

Concurrent Readers

Reader1
Reader2
Reader3
Reader4

All readers can execute simultaneously.

Better Performance

Suitable when:

Many Reads
Few Writes

Improved Scalability

Multiple CPUs can read shared data concurrently.

Disadvantages

Writer Starvation

If readers continuously access the lock:

Reader1
Reader2
Reader3
Reader4

...

Writer may wait indefinitely.

Writer -> Waiting

This is called:

Writer Starvation

When to Use RW Locks?

Use RW Locks when:

- ✓ Read operations are frequent
- ✓ Write operations are rare
- ✓ Multiple readers need simultaneous access
- ✓ Read performance is important

Examples:

Device Configuration
Routing Tables
Kernel Statistics
Status Information
Lookup Tables

When Not to Use RW Locks?

Avoid RW Locks when:

Reads $\approx$ Writes

or

Mostly Writes

In such cases:

Mutex
Spinlock

are generally better choices.

Why use RW Locks?

To allow multiple readers to access shared data concurrently while still ensuring exclusive access for writers.

Difference Between Read Lock and Write Lock?

Read Lock:

Multiple readers allowed

Write Lock:

Only one writer allowed

No readers allowed

What is Writer Starvation?

A situation where continuous readers prevent a waiting writer from acquiring the lock.

Types

rwlock_t -> Spin Based

rw_semaphore -> Sleep Based

APIs

read_lock()

read_unlock()

write_lock()

write_unlock()

down_read()

up_read()

down_write()

up_write()

Problem

Writer Starvation

Easy Memory Trick

Many Reads + Few Writes

↓

RW Lock

Short Critical Section

↓

RW Spinlock

Long Critical Section

↓
RW Semaphore

RW-SPINLOCK OUTPUT:

```
vboxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SPINLOCK$ make
make -C /lib/modules/6.8.0-124-generic/build M=/home/vboxuser/synchronizaation/READWRITELOCKS/RW_SPINLOCK
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
You are using:          gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
CC [M] /home/vboxuser/synchronizaation/READWRITELOCKS/RW_SPINLOCK/rw_spinlock_demo.o
MODPOST /home/vboxuser/synchronizaation/READWRITELOCKS/RW_SPINLOCK/Module.symvers
CC [M] /home/vboxuser/synchronizaation/READWRITELOCKS/RW_SPINLOCK/rw_spinlock_demo.mod.o
LD [M] /home/vboxuser/synchronizaation/READWRITELOCKS/RW_SPINLOCK/rw_spinlock_demo.ko
BTF [M] /home/vboxuser/synchronizaation/READWRITELOCKS/RW_SPINLOCK/rw_spinlock_demo.ko
Skipping BTF generation for /home/vboxuser/synchronizaation/READWRITELOCKS/RW_SPINLOCK/rw_spinlock_demo.ko
f vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
vboxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SPINLOCK$ ls
Makefile      Module.symvers rw_spinlock_demo.c  rw_spinlock_demo.mod  rw_spinlock_demo.mod.o
modules.order README.md      rw_spinlock_demo.ko rw_spinlock_demo.mod.c rw_spinlock_demo.o
vboxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SPINLOCK$ sudo insmod rw_spinlock_demo.ko
vboxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SPINLOCK$ sudo dmesg | tail -15
[ 8037.179793] Reader-2: shared_data = 1
[ 8038.204266] Reader-2: shared_data = 1
[ 8038.204624] Reader-1: shared_data = 1
[ 8038.460317] Writer: Updated shared_data = 2
[ 8039.227957] Reader-2: shared_data = 2
[ 8039.227972] Reader-1: shared_data = 2
[ 8040.251888] Reader-2: shared_data = 2
[ 8040.253012] Reader-1: shared_data = 2
[ 8041.275465] Reader-2: shared_data = 2
[ 8041.275793] Reader-1: shared_data = 2
[ 8042.300960] Reader-1: shared_data = 2
[ 8042.301256] Reader-2: shared_data = 2
[ 8043.324029] Reader-1: shared_data = 2
[ 8043.324047] Reader-2: shared_data = 2
[ 8043.579911] Writer: Updated shared_data = 3
vboxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SPINLOCK$ sudo dmesg | tail -20
[ 8049.468184] Reader-2: shared_data = 4
[ 8050.492395] Reader-1: shared_data = 4
[ 8050.492418] Reader-2: shared_data = 4
```

RW-SEMAPHORE OUTPUT:

```

make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
boxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SEMAPHORE$ ls
Makefile      Module.symvers  rw_semaphore_demo.c  rw_semaphore_demo.mod  rw_semaphore_demo.o
modules.order  README.md       rw_semaphore_demo.ko  rw_semaphore_demo.mod.c  rw_semaphore_demo.o
boxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SEMAPHORE$ sudo insmod rw_semaphore_demo.o
boxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SEMAPHORE$ sudo dmesg | tail -20
8291.585270] Reader-1: shared_data = 51
8291.585346] Reader-2: shared_data = 51
8292.478135] Reader-2: shared_data = 1
8292.478147] Reader-1: shared_data = 1
8292.605866] Reader-2: shared_data = 51
8292.606107] Reader-1: shared_data = 51
8293.504884] Reader-1: shared_data = 1
8293.504896] Reader-2: shared_data = 1
8293.634935] Reader-2: shared_data = 51
8293.635064] Reader-1: shared_data = 51
8294.462541] Writer: Updated shared_data = 52
8294.526344] Writer: Updated shared_data = 2
8294.654034] Reader-2: shared_data = 52
8294.654043] Reader-1: shared_data = 52
8295.678041] Reader-1: shared_data = 52
8295.678259] Reader-2: shared_data = 52
8296.702319] Reader-2: shared_data = 52
8296.702410] Reader-1: shared_data = 52
8297.535158] Reader-1: shared_data = 2
8297.535234] Reader-2: shared_data = 2
boxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SEMAPHORE$ sudo rmmod rw_spinlock_demo
boxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SEMAPHORE$ sudo dmesg | tail -15
8325.182670] Writer: Updated shared_data = 58
8327.934526] Reader-2: shared_data = 8
8327.934958] Reader-1: shared_data = 8
8328.958394] Reader-1: shared_data = 8
8328.958441] Reader-2: shared_data = 8
8329.983048] Reader-2: shared_data = 8
8329.983448] Writer: Updated shared_data = 9
8330.303192] RW Spinlock Module Unloaded
8332.991445] Reader-2: shared_data = 9
8332.991941] Reader-1: shared_data = 9
8334.014593] Reader-2: shared_data = 9
8334.030664] Reader-1: shared_data = 9
8335.039079] Reader-1: shared_data = 9
8335.039163] Reader-2: shared_data = 9
8335.039189] Writer: Updated shared_data = 10
boxuser@ubuntu2204:~/synchronizaation/READWRITELOCKS/RW_SEMAPHORE$ █

```

Q: Can a read lock be upgraded directly to a write lock?

Answer:

No. Linux Read-Write locks do not support automatic lock upgrading. If a thread holding a read lock tries to acquire a write lock, it will deadlock because the write lock waits for all readers to release the lock, including the current thread.

downgrade write()

Read writer semaphores have a unique method which is not present in reader-writer spinlocks.

This function automatically convert an acquired write lock to a read lock

Use case: used in the situation where writer lock is needed for a quick change, followed by longer period of read -only access.